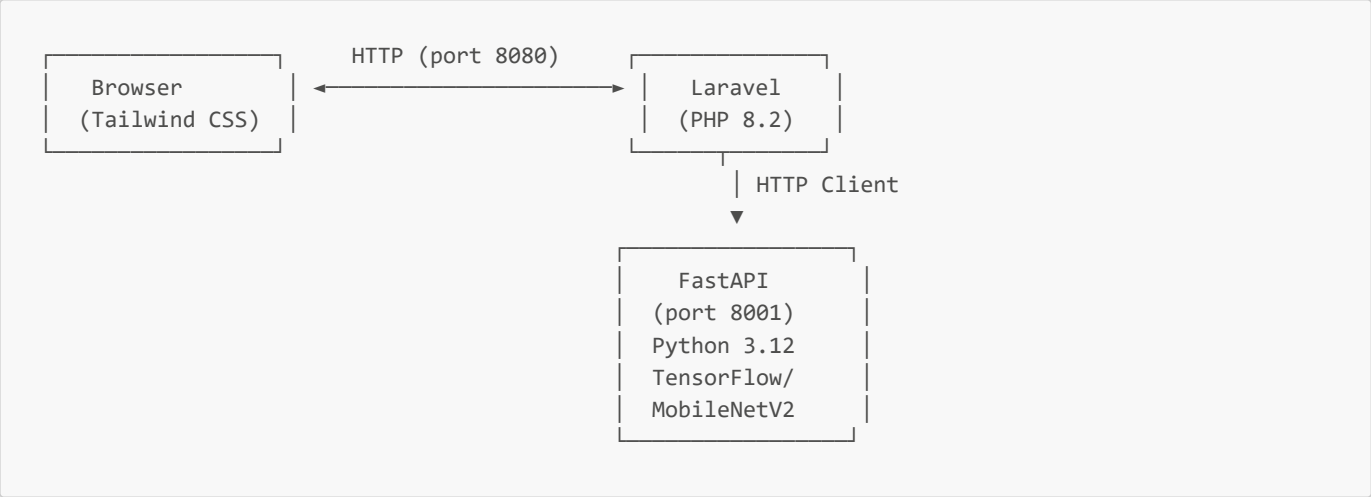


MediScan AI — Dokumen Dasar Proyek

Mata Kuliah: Kecerdasan Buatan **Judul:** Klasifikasi Kanker Payudara pada Citra USG Menggunakan MobileNetV2 **Tahun:** 2026

1. Arsitektur Sistem



Komponen	Teknologi	Port
Frontend & Backend Bisnis	Laravel 12 (PHP 8.2), Blade, Tailwind CSS	8080
AI Service	FastAPI (Python 3.12), TensorFlow/Keras	8001
Database	MySQL 8.0	3306

2. Dataset

Sumber: Breast Ultrasound Images Dataset (BUSI) — Kaggle **Referensi:** Al-Dhabyani et al., Data in Brief, 2020

Kelas	Jumlah Gambar
Benign (Jinak)	891 file (asli + mask)
Malignant (Ganas)	421 file (asli + mask)
Normal	266 file (asli + mask)
Total	1.578 file

- Resolusi: ~500×500 piksel, format PNG
- Setiap gambar memiliki mask/ground truth
- Filter otomatis: file mengandung kata "mask" / "label" / "annotation" / "gt" diexclude sebelum training

3. Model — MobileNetV2 (arsitektur & training)

Arsitektur

```

Input (224×224×3)
→ MobileNetV2 (weights='imagenet', include_top=False, pooling='avg')
→ Dense(256, ReLU) → Dropout(0.5)
→ Dense(128, ReLU) → Dropout(0.3)
→ Dense(3, Softmax) [benign, malignant, normal]

```

Training 2 Fase

Fase	Layer Base Model	Learning Rate	Epoch
1 — Frozen	Semua dibekukan	1×10^{-3}	50% total
2 — Fine-Tuning	Layer 120+ aktif	1×10^{-5}	50% total

Penanganan Data Tidak Seimbang

- **Class Weight:** $\text{total_samples} / (\text{n_classes} \times \text{count_per_class})$
- **Augmentasi:** rotation, shift, shear, zoom, horizontal_flip
- **Focal Loss** dengan gamma=2.0, alpha=0.25 (fokus pada kelas malignant)

Loss Function: SparseFocalLoss

ai-service/app/trainer.py:36-58

```

class SparseFocalLoss(tf.keras.losses.Loss):
    def __init__(self, gamma=2.0, alpha=0.25, from_logits=False, **kwargs):
        super().__init__(**kwargs)
        self.gamma = gamma
        self.alpha = alpha
    def call(self, y_true, y_pred):
        epsilon = tf.keras.backend.epsilon()
        y_pred = tf.clip_by_value(y_pred, epsilon, 1.0 - epsilon)
        y_true = tf.cast(y_true, tf.int32)
        y_true_one_hot = tf.one_hot(y_true, tf.shape(y_pred)[-1])
        cross_entropy = -y_true_one_hot * tf.math.log(y_pred)
        focal_weight = tf.pow(1.0 - y_pred, self.gamma) * y_true_one_hot
        focal_loss = self.alpha * focal_weight * cross_entropy
        return tf.reduce_sum(focal_loss, axis=-1)

```

Pembuatan Arsitektur Model (Fase 1 — Frozen)

ai-service/app/trainer.py:258-274

```

base_model = MobileNetV2(
    weights="imagenet", include_top=False, pooling="avg",
    input_shape=(image_size, image_size, 3)
)
base_model.trainable = False
x = base_model.output
x = Dense(256, activation="relu")(x)
x = tf.keras.layers.Dropout(0.5)(x)
x = Dense(128, activation="relu")(x)
x = tf.keras.layers.Dropout(0.3)(x)
outputs = Dense(n_classes, activation="softmax")(x)
model = Model(inputs=base_model.input, outputs=outputs)
model.compile(
    optimizer=Adam(learning_rate=1r),

```

```

        loss=SparseFocalLoss(gamma=2.0, alpha=0.25),
        metrics=["accuracy"]
    )

```

Fase 2 — Fine-Tuning

ai-service/app/trainer.py:286-294

```

base_model.trainable = True
for layer in base_model.layers[:120]:
    layer.trainable = False
model.compile(
    optimizer=Adam(learning_rate=lr * 0.1),
    loss=SparseFocalLoss(gamma=2.0, alpha=0.25),
    metrics=["accuracy"]
)

```

Class Weight untuk Data Tidak Seimbang

ai-service/app/trainer.py:194-200

```

def _compute_class_weights(train_gen, class_names):
    class_counts = np.bincount(train_gen.classes)
    total = len(train_gen.classes)
    n_classes = len(class_names)
    weights = total / (n_classes * class_counts.astype(float))
    class_weight_dict = {i: float(w) for i, w in enumerate(weights)}
    return class_weight_dict

```

Data Augmentation (ImageDataGenerator)

ai-service/app/trainer.py:221-232

```

train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input,
    rotation_range=20,
    width_shift_range=0.15,
    height_shift_range=0.15,
    shear_range=0.15,
    zoom_range=0.15,
    brightness_range=(0.8, 1.2),
    horizontal_flip=True,
    fill_mode="nearest",
    validation_split=val_split
)

```

ProgressCallback — Tracking Epoch Real-Time

ai-service/app/trainer.py:149-176

```

class ProgressCallback(tf.keras.callbacks.Callback):
    def __init__(self, job_id, epoch_offset=0):
        super().__init__()
        self.job_id = job_id

```

```
self.epoch_offset = epoch_offset
def on_epoch_end(self, epoch, logs=None):
    job = jobs.get(self.job_id)
    if job and job.get("cancel_requested"):
        self.model.stop_training = True
        return
    logs = logs or {}
    display_epoch = epoch + 1 + self.epoch_offset
    _update_job(self.job_id, current_epoch=display_epoch)
    ep = {"epoch": display_epoch, "accuracy": float(logs.get("accuracy", 0)),
          "loss": float(logs.get("loss", 0)),
          "val_accuracy": float(logs.get("val_accuracy", 0)),
          "val_loss": float(logs.get("val_loss", 0))}
    if self.job_id in jobs:
        jobs[self.job_id].setdefault("epochs", []).append(ep)
```

Hasil Akhir

Metrik	Nilai
Akurasi Validasi	81.11%
Loss Validasi	0.4499

4. Konfigurasi Sistem

ai-service/app/config.py:1-18

```
STORAGE_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..", "storage"))
MODELS_DIR = os.path.join(STORAGE_DIR, "models")
DATASETS_DIR = os.path.join(STORAGE_DIR, "datasets")
PREDICTIONS_DIR = os.path.join(STORAGE_DIR, "predictions")
MAX_DATASET_SIZE_MB = 500
VALID_EXTENSIONS = {".jpg", ".jpeg", ".png"}
MASK_KEYWORDS = ["mask", "label", "annotation", "gt"]
MIN_IMAGES_PER_CLASS = 20
WARNING_IMAGES_PER_CLASS = 50
IMG_SIZE = (224, 224)
RANDOM_SEED = 42
```

5. Database (15 migration files)

laravel/database/migrations/

Tabel	File	Kolom Kunci
admins	054006_create_admins_table	name, email, password
training_jobs	054007_create_training_jobs_table	status (enum: pending/validating/extracting/training/completed/failed/cancelled), current_epoch, total_epoch, accuracy_result, loss_result, precision_result, recall_result, f1_score_result, log, epoch_history (JSON)
ai_models	054008_create_ai_models_table	name, class_names (JSON), file_path, accuracy, loss, is_active (boolean), model_id (unique), notes
predictions	054009_create_predictions_table	image_path, original_name, predicted_class, confidence, probabilities (JSON), ai_model_id, patient_name, patient_age,

Tabel	File	Kolom Kunci
		grad_cam_path, model_label

6. Endpoint FastAPI

ai-service/app/main.py

Method	Endpoint	Fungsi
POST	/train	Upload dataset ZIP, mulai training background
GET	/train/{job_id}/status	Polling progress training real-time
GET	/train/{job_id}/log	Ambil log training mentah
POST	/train/{job_id}/cancel	Batalkan training
POST	/train/{job_id}/activate	Aktivasi model hasil training
POST	/predict	Prediksi 1 gambar USG + Grad-CAM
GET	/models/active	Ambil model yang aktif
GET	/models/list	Daftar semua model
POST	/models/{model_id}/activate	Aktivasi model spesifik
DELETE	/models/{model_id}	Hapus model + semua filenya
POST	/models/register	Register model .keras eksternal
GET	/files/confusion_matrix/{model_id}	Gambar confusion matrix
GET	/files/confusion_matrix_data/{model_id}	Data JSON confusion matrix
GET	/files/gradcam/{filename}	Gambar Grad-CAM
GET	/health	Health check

Endpoint Prediksi + Grad-CAM

ai-service/app/main.py:290-324

```
@app.post("/predict")
async def predict(file: UploadFile = File(...)):
    ext = os.path.splitext(file.filename)[1].lower()
    if ext not in VALID_EXTENSIONS:
        raise HTTPException(400, f"Invalid file type")
    model, class_names, model_id = _load_active_model()
    if model is None:
        raise HTTPException(400, "No active model available")
    image_bytes = await file.read()
    original_pil = Image.open(io.BytesIO(image_bytes)).convert("RGB")
    image = original_pil.resize(IMG_SIZE)
    img_array = img_to_array(image)
    img_array = np.expand_dims(img_array, axis=0)
    img_array = preprocess_input(img_array)
    predictions = model.predict(img_array, verbose=0)[0]
    predicted_idx = int(np.argmax(predictions))
    confidence = float(predictions[predicted_idx])
    predicted_class = class_names[predicted_idx]
    probabilities = {class_names[i]: float(predictions[i]) for i in range(len(class_names))}
    grad_cam_filename = _generate_gradcam(model, img_array, predicted_idx, original_pil)
    return {
```

```

        "predicted_class": predicted_class,
        "confidence": confidence,
        "probabilities": probabilities,
        "grad_cam_url": f"/files/gradcam/{grad_cam_filename}",
    }

```

Generate Grad-CAM Heatmap

ai-service/app/main.py:86-110

```

def _generate_gradcam(model, img_array, predicted_idx, original_pil):
    last_conv = model.get_layer('out_relu')
    grad_model = tf.keras.models.Model(
        inputs=model.inputs,
        outputs=[last_conv.output, model.output]
    )
    with tf.GradientTape() as tape:
        conv_out, preds = grad_model(img_array)
        loss = preds[:, predicted_idx]
    grads = tape.gradient(loss, conv_out)
    pooled = tf.reduce_mean(grads, axis=(0, 1, 2))
    heatmap = tf.reduce_sum(tf.multiply(pooled, conv_out[0]), axis=-1)
    heatmap = tf.maximum(heatmap, 0) / (tf.reduce_max(heatmap) + tf.keras.backend.epsilon())
    heatmap = heatmap.numpy()
    heatmap = cv2.resize(heatmap, original_pil.size)
    heatmap = np.uint8(255 * heatmap)
    heatmap_color = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)
    original_bgr = cv2.cvtColor(np.array(original_pil), cv2.COLOR_RGB2BGR)
    overlay = cv2.addWeighted(original_bgr, 0.6, heatmap_color, 0.4, 0)
    filename = f"gradcam_{uuid.uuid4()}.png"
    cv2.imwrite(os.path.join(PREDICTIONS_DIR, filename), overlay)
    return filename

```

Endpoint Training (Background Thread)

ai-service/app/main.py:124-148

```

@app.post("/train")
async def create_training(
    file: UploadFile = File(...),
    epochs: int = Form(10),
    validation_split: float = Form(0.3),
    batch_size: int = Form(32),
    image_size: int = Form(224),
    learning_rate: float = Form(0.0001),
):
    zip_path = os.path.join(DATASETS_DIR, f"{uuid.uuid4()}_{file.filename}")
    content = await file.read()
    with open(zip_path, "wb") as f:
        f.write(content)
    job_id = start_training(
        zip_path=zip_path, epochs=epochs,
        validation_split=validation_split, batch_size=batch_size,
        image_size=image_size, learning_rate=learning_rate,
    )
    return {"job_id": job_id, "status": "pending"}

```

Validasi Dataset (Filter Mask Files)

ai-service/app/trainer.py:107-147

```
def _validate_dataset(job_id, dataset_path):
    classes = sorted([d for d in os.listdir(dataset_path)
                      if os.path.isdir(os.path.join(dataset_path, d))])
    if len(classes) < 2:
        raise ValueError(f"Minimal 2 folder kelas, ditemukan {len(classes)}")
    for cls in classes:
        cls_path = os.path.join(dataset_path, cls)
        all_files = sorted(os.listdir(cls_path))
        for f in all_files:
            ext = os.path.splitext(f)[1].lower()
            if ext not in VALID_EXTENSIONS:
                skipped += 1; continue
            f_lower = f.lower()
            if any(kw in f_lower for kw in MASK_KEYWORDS):
                skipped += 1; continue # Filter file mask!
            valid_files.append(f)
        if len(valid_files) < MIN_IMAGES_PER_CLASS:
            raise ValueError(f"Kelas '{cls}' hanya punya {count} gambar")
```

Webhook Notifikasi ke Laravel

ai-service/app/trainer.py:361-389

```
laravel_url = os.environ.get(
    "LARAVEL_WEBHOOK_URL",
    "http://localhost:8080/api/training/webhook"
)
resp = requests.post(laravel_url, json={
    "job_id": job_id,
    "status": "Completed",
    "accuracy": float(metrics_dict["accuracy"]),
    "loss": float(val_loss),
    "precision": float(metrics_dict["precision"]),
    "recall": float(metrics_dict["recall"]),
    "f1_score": float(metrics_dict["f1_score"]),
    "epoch_history": epoch_history,
    "current_epoch": current_epoch,
    "total_epoch": total_epoch,
    "model_id": job_data.get("model_id", ""),
}, timeout=5)
```

7. Laravel — HTTP Client ke FastAPI (AiService)

laravel/app/Services/AiService.php:8-207

Menjembatani Laravel dengan FastAPI via HTTP:

```
class AiService
{
    protected string $baseUrl;
    protected int $timeout;

    public function __construct()
    {
        $this->baseUrl = rtrim(env('AI_SERVICE_URL', 'http://localhost:8001'), '/');
```

```

        $this->timeout = 5;
    }

    // Upload dataset ZIP → FastAPI POST /train
    public function uploadDataset($zipPath, $originalName, ...): array
    {
        $response = Http::timeout(300)->attach(
            'file', file_get_contents($zipPath), $originalName
        )->post("{ $this->baseUrl}/train", [
            'epochs' => $epochs, 'validation_split' => $validationSplit,
            'batch_size' => $batchSize, 'image_size' => $imageSize,
            'learning_rate' => $learningRate,
        ]);
        return $response->json();
    }

    // Prediksi gambar → FastAPI POST /predict
    public function predict($imagePath, $originalName): array
    {
        $response = Http::timeout(30)->attach(
            'file', file_get_contents($imagePath), $originalName
        )->post("{ $this->baseUrl}/predict");
        return $response->json();
    }

    // Daftar model → FastAPI GET /models/list
    public function listModels(): array { ... }
    // Aktifkan model → FastAPI POST /models/{id}/activate
    public function activateSpecifcModel($modelId): array { ... }
    // Hapus model → FastAPI DELETE /models/{id}
    public function deleteModel($modelId): array { ... }
}

```

8. Laravel — PredictionController

laravel/app/Http/Controllers/PredictionController.php:29-104

```

public function predict(Request $request)
{
    $request->validate([
        'image' => 'required|image|mimes:jpeg,png,jpg|max:10240',
        'patient_name' => 'required|string|max:255',
        'patient_age' => 'required|integer|min:1|max:150',
    ]);

    $activeModel = $this->aiService->getActiveModel();
    if (!$activeModel['active'] ?? false) {
        return back()->with('error', 'Belum ada model aktif.');
```



```

        'patient_age' => (int) $request->input('patient_age'),
        'image_path' => $imagePath,
        'original_name' => $image->getClientOriginalName(),
        'predicted_class' => $result['predicted_class'],
        'confidence' => $result['confidence'],
        'probabilities' => $result['probabilities'] ?? [],
    ];

    // Download Grad-CAM dari FastAPI ke local storage
    if (!empty($result['grad_cam_url'])) {
        $gradCamContents = @file_get_contents($result['grad_cam_url']);
        $gradCamPath = 'predictions/gradcam_' . uniqid() . '.png';
        Storage::disk('public')->put($gradCamPath, $gradCamContents);
        $prediction->update(['grad_cam_path' => $gradCamPath]);
    }
    return response()->json($result);
}

```

9. Laravel — Routes

laravel/routes/web.php:1-40

```

// Publik
Route::get('/', [LandingController::class, 'index']);
Route::get('/tes', [PredictionController::class, 'index']);
Route::post('/tes/predict', [PredictionController::class, 'predict']);

// Webhook dari FastAPI
Route::post('/api/training/webhook', [AdminTrainingController::class, 'webhook']);

// Admin (login required)
Route::prefix('admin')->group(function () {
    Route::get('/login', [AdminLoginController::class, 'showLoginForm']);
    Route::post('/login', [AdminLoginController::class, 'login']);

    Route::middleware('auth:admin')->group(function () {
        Route::get('/dashboard', [AdminDashboardController::class, 'index']);
        Route::get('/models', [AdminModelController::class, 'index']);
        Route::post('/models/register', [AdminModelController::class, 'store']);
        Route::post('/models/{modelId}/activate', [AdminModelController::class, 'activate']);
        Route::delete('/models/{modelId}', [AdminModelController::class, 'destroy']);
        Route::get('/training', [AdminTrainingController::class, 'index']);
        Route::post('/training', [AdminTrainingController::class, 'upload']);
        Route::post('/training/{jobId}/cancel', [AdminTrainingController::class, 'cancel']);
        Route::get('/training/{jobId}', [AdminTrainingController::class, 'show']);
        Route::get('/predictions', [AdminPredictionController::class, 'index']);
        Route::delete('/predictions/{id}', [AdminPredictionController::class, 'destroy']);
    });
});

```

10. Fitur Unggulan

1. **Grad-CAM Visualization** — Heatmap area gambar yang menjadi fokus model saat prediksi (layer `out_relu` MobileNetV2) — `main.py:86-110`
2. **SparseFocalLoss** — Loss function custom dengan $\gamma=2.0$, $\alpha=0.25$, fokus pada kelas malignant — `trainer.py:36-58`
3. **Image Gatekeeping** — Deteksi otomatis apakah gambar adalah citra USG yang valid — `main.py`
4. **Training Real-Time** — Polling progress tiap 3-5 detik via ProgressCallback + JS fetch — `trainer.py:149-176`
5. **Confusion Matrix Dinamis** — Akurasi global, sensitivitas per kelas, rekomendasi optimasi — `trainer.py:394-416`

6. **Filter Dataset** — Otomatis skip file mask/label/annotation dari dataset ZIP — [trainer.py:107-147](#)

7. **Kamera Capture** — Prediksi langsung dari kamera HP di halaman [/tes](#)

11. Halaman Web

Publik (tanpa login)

URL	Halaman
/	Landing page dark futuristic
/tes	Upload gambar + hasil prediksi + Grad-CAM

Admin (login)

URL	Halaman
/admin/login	Login
/admin/dashboard	Ringkasan (jumlah model, prediksi, training)
/admin/models	CRUD model + aktivasi + confusion matrix
/admin/training	Upload dataset + mulai training
/admin/training/{id}	Progress training real-time
/admin/predictions	Riwayat prediksi + detail + download

Kredensial Default: [admin@medical-classifier.com](#) / [password](#)

12. Status Pengerjaan

Komponen	Status
Laravel: Database, Auth, Migration	✓ Selesai
Laravel: Landing, Prediksi Publik	✓ Selesai
Laravel: Admin Dashboard, Models, Training, Predictions	✓ Selesai
FastAPI: Training pipeline (2 fase)	✓ Selesai
FastAPI: Prediksi + Grad-CAM	✓ Selesai
FastAPI: Gatekeeper citra USG	✓ Selesai
FastAPI: Model management	✓ Selesai
Testing & Verifikasi Akhir	🕒 Sebagian

13. Cara Menjalankan

```
# Terminal 1 – FastAPI (port 8001)
cd ai-service
python -m uvicorn app.main:app --host 0.0.0.0 --port 8001 --reload

# Terminal 2 – Laravel (port 8080)
cd laravel
php -d upload_max_filesize=500M -d post_max_size=500M artisan serve --port=8080
```

Akses: <http://localhost:8080> (Laravel) | <http://localhost:8001/docs> (Swagger FastAPI)

14. Referensi

1. Al-Dhabyani W, Gomaa M, Khaled H, Fahmy A. Dataset of breast ultrasound images. Data in Brief. 2020 Feb;28:104863.
2. Sandler M, Howard A, Zhu M, et al. MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR 2018.
3. Selvaraju RR, Cogswell M, Das A, et al. Grad-CAM: Visual Explanations from Deep Networks. ICCV 2017.